TRAFFIX

GOOGLE AI STUDIO FULL-STACK BUILD PROMPT

Copy-ready instructions for generating a complete working application — not just a frontend design.

PROJECT	Intelligent Traffic Management and Route Optimization System
PROBLEM	PS 01 – Intelligent Traffic Management and Route Optimization System
DELIVERABLE	Frontend + backend + APIs + database + validation + traffic algorithms + tests

HOW TO USE THIS PDF

Upload your original PS 01 problem statement and the TRAFFIX master specification into Google AI Studio. Then paste the full prompt in this document. Ask it to create actual project files, run the application, test it and fix errors. Do not accept a design-only response.

NON-NEGOTIABLE RULE: Every major screen, button, form, filter and dashboard action must connect to backend logic and persistent data. Never mark a feature complete unless it has been implemented and tested.

COPY-READY PROMPT FOR GOOGLE AI STUDIO

START OF PROMPT

You are acting as a senior full-stack software engineering team. Build **TRAFFIX – Intelligent Traffic Management and Route Optimization System** according to the attached PS 01 problem statement and TRAFFIX master specification.

I need a **complete, runnable full-stack application**, not merely a frontend design, static prototype, mock dashboard, code snippets or disconnected pages. Generate actual source files and implement the complete flow from frontend interaction to backend processing and database persistence.

Follow the technology stack and feature requirements in the attached specification. Do not silently replace or omit mandatory requirements. If a live service is unavailable, implement a clearly labelled simulation or documented fallback. Never present simulated traffic as live data.

1. Understand the problem, users and expected solution

- Analyse PS 01 and list the problems, user roles, mandatory features, technical expectations and deliverables.
- Support commuters, traffic administrators, traffic-control operators and authorized emergency-vehicle operators.
- Create a requirement checklist mapping each PS 01 requirement to its screen, backend service, API, database entities and test.
- Clearly distinguish required features from optional enhancements.

2. Generate the complete application

- Create a responsive, accessible frontend with a consistent professional traffic-operations design.
- Create the backend server, controllers/routes, service layer, database schema, migrations, seed data and configuration files.
- Create the prediction service if required by the selected architecture.
- Generate actual source files, project structure, package scripts, dependency files, .env.example and README.
- Do not leave essential modules as placeholders or TODO comments.

3. Generate real backend logic, APIs and validation

- Implement server-side logic for authentication, route calculation, incident reporting, duplicate detection, severity assessment, network impact, congestion prediction, signal optimization, emergency routing and administration.
- Expose documented REST APIs for all major features. Document HTTP method, endpoint, purpose, request, response and status codes.
- Validate incoming data on the server, including required fields, types, ranges, coordinates and user permissions.
- Return consistent success and error responses. Handle validation, not-found, unauthorized, forbidden, conflict, database and unexpected errors.
- Add a health-check endpoint and API documentation, preferably Swagger/OpenAPI.
- Keep important business logic on the server; do not trust values supplied by the frontend.

4. Database integration and persistence

- Create the actual database schema with primary keys, foreign keys, relationships, indexes and constraints.
- Persist users, roles, incidents, review actions, road/junction data, traffic observations, predictions, simulations, emergency trips and audit/history records.
- Implement migrations and development seed data. Mark test accounts and sample records as development-only.
- Retrieve submitted records through APIs. Data must remain after page refresh and server restart.
- Use safe database access and transactions where appropriate. Never hardcode database credentials.

5. Connect every frontend interaction to the backend

- Create a typed API client/service layer and configure the backend URL through environment variables.

- Connect every primary button, form, search, filter, admin decision and simulation control to the correct API or clearly explained navigation action.
- Registration/login must call authentication APIs; incident submission must call the incident API; admin review must update stored data; route search, prediction and signal controls must call backend services.
- Show loading, success, empty and error states. Prevent accidental duplicate submissions.
- Display the actual server response. Never show success before the server confirms it.
- Do not substitute hardcoded frontend arrays or fake API responses for backend data. Seeded demo data is acceptable only when stored in the database and labelled as sample data.

6. Implement all PS 01 traffic features

- Route search: source/destination, multiple alternatives, shortest/fastest/recommended comparison, distance, ETA and multi-factor scoring.
- Incident reporting: accidents, roadblocks and congestion; location and details; map display; administrator verification, rejection and resolution.
- Congestion prediction: 15-minute and 30-minute forecasts using available historical/simulated traffic, time, road capacity, weather and incident inputs; show risk levels and data provenance.
- Duplicate/severity analysis: identify likely duplicate reports and estimate severity using explainable rules or a documented model.
- Network impact: analyse connected roads, simulate road/lane closures, calculate affected paths and reroute where possible.
- Adaptive signals: use simulated vehicle density and queue length to recommend bounded timings; compare waiting time and queue metrics before/after.
- Emergency priority: calculate an authorized ambulance/fire-service route, identify affected junctions and compare normal versus priority ETA. Label signal actions as simulations unless connected to authorized infrastructure.
- Real-time updates: deliver authorized incident, route, trip and operations updates with reconnect and error handling.

7. Security and reliability

- Hash passwords securely; never store plain-text passwords.
- Implement authentication and backend-enforced role-based authorization. Hiding a frontend button is not security.
- Protect administrator, traffic-control and emergency-operator endpoints.
- Use environment variables for secrets and placeholders only in .env.example.
- Configure CORS, secure headers, request-size limits, rate limiting and safe error logging.
- Never expose stack traces, secrets or sensitive user data in API responses.

8. Test the full application

- Register a user, log in, retrieve the profile and verify invalid credentials are rejected.
- Submit a valid incident, retrieve it from the database, refresh and verify it remains available.
- Submit invalid incident data and confirm server-side validation rejects it.
- Try admin-only APIs as a normal user and confirm access is denied.
- Submit a nearby duplicate incident and verify it is flagged.
- Verify admin review and resolution are persisted.
- Search routes; test a closure and verify route recalculation.
- Run prediction, signal optimization and emergency-routing scenarios and inspect outputs.
- Test missing configuration, unavailable backend/database and unexpected errors. Show understandable frontend error states.

9. Work incrementally and keep the project runnable

- Build in phases: (1) architecture/project structure, (2) database/backend foundation, (3) authentication/APIs, (4) frontend integration, (5) traffic algorithms/advanced features, (6) real-time updates/analytics, (7) tests/debugging/documentation.
- At each phase, list files created or changed, commands run, tests performed and known issues.
- Run relevant tests and fix errors before proceeding. Never claim a test passed if it was not run.
- If the context limit is reached, continue from the last completed phase without replacing working code or omitting earlier requirements.

10. Final deliverables and student demonstration

- Provide complete source code and exact local setup/run instructions.
- Provide architecture explanation, database relationship summary, API list and sample request/response examples.
- Provide test results marked passed/failed/not-run and explain limitations.
- Provide a live demo sequence: registration, incident submission, admin review, route search, road closure, prediction, signal simulation and emergency priority routing.
- Explain frontend → API → backend service → database → API response → frontend in simple student-friendly language.
- Prepare likely judge questions and accurate answers. Distinguish implemented features, simulations and future integrations.

FINAL ACCEPTANCE RULE

Do not declare TRAFFIX complete until frontend, backend and database are connected, key workflows have been exercised, errors have been fixed where possible, and remaining limitations are documented. A polished interface alone is not an acceptable deliverable.

END OF PROMPT

PRACTICAL INSTRUCTION IF AI STUDIO STARTS WITH UI ONLY

Stop it and say: “Do not continue designing screens. First create and run the backend, database schema, migrations and health-check API. Then connect the frontend to those working endpoints.”

Do not paste private API keys, passwords or real personal data into the prompt. Use environment variables and development-only sample accounts.